

Chapter 11: Standalone Components

1. Overview

For seven years, every Angular component, directive, and pipe had to be declared inside exactly one `NgModule`. `NgModule` was the unit of compilation, the unit of dependency injection scoping, and the unit of lazy loading. It worked, but it also became the single biggest source of boilerplate and beginner confusion in the framework: `declarations`, `exports`, `imports`, `entryComponents` (before Ivy), circular module dependencies, "why is my component not found" errors caused by forgetting an export, and the notorious `SharedModule` grab-bag that every feature module imported "just in case."

Starting in **Angular 14** (developer preview) and stabilizing in **Angular 15**, Angular introduced the **standalone API**: components, directives, and pipes that declare their own dependencies directly, with no enclosing `NgModule` required. In **Angular 17**, standalone became the **default** for new projects generated by the CLI (`standalone: true` is implied — you no longer even write the flag). In **Angular 19**, the flag was removed entirely from schematics output because standalone is now simply "how components work" unless you opt out.

1.1 NgModule Architecture vs Standalone Architecture

Aspect	NgModule-based	Standalone
Declaring a component's template dependencies	Done at the <i>module</i> level via <code>declarations</code> + <code>imports</code> — a component doesn't say what it needs, its module does	Done at the <i>component</i> level via its own <code>imports</code> array — self-describing
Bootstrapping	<code>platformBrowserDynamic().bootstrapModule(AppModule)</code> with a <code>bootstrap: [AppComponent]</code> array	<code>bootstrapApplication(AppComponent, config)</code> — no root module at all
Lazy loading a feature	<code>loadChildren: () => import('./feature/feature.module').then(m => m.FeatureModule)</code>	<code>loadChildren: () => import('./feature/feature.routes').then(m => m.FEATURE_ROUTES)</code> or <code>loadComponent</code> directly
Providing services	<code>providers</code> array on a module (often <code>CoreModule</code> imported once in <code>AppModule</code>)	<code>providers</code> array in <code>bootstrapApplication</code> , on a route (<code>providers</code> on <code>Route</code>), or <code>providedIn: 'root'</code> on the service itself
Sharing common directives/pipes	<code>SharedModule</code> re-exporting <code>CommonModule</code> , custom pipes, directives	Import the directive/pipe/component directly where used; no aggregator module needed
Mental model	Two-level graph: modules import modules, components live inside modules	Single-level graph: components import components/directives/pipes directly
Tree-shaking granularity	Coarse — often module-level, easy to pull in unused code transitively	Fine — each component's <code>imports</code> array is exactly what the compiler needs, better dead-code elimination
Boilerplate for a simple feature	New component + update <code>declarations</code> in some module	New component, nothing else
Learning curve for beginners	Must understand modules <i>before</i> understanding "why isn't <code>*ngIf</code> found"	Import <code>NgIf</code> / <code>CommonModule</code> directly where used, immediately visible

1.2 Why Angular Moved Away from NgModules

- 1. Reduce boilerplate and cognitive overhead.** NgModules require developers to reason about two separate graphs (module graph and component graph) to answer a simple question: "what can this component's template use?" Standalone collapses this to one graph.
- 2. Better alignment with how developers actually think.** Most confusion for newcomers was "why

doesn't `*ngFor` work" — answered by "you forgot to import `CommonModule`" in some *other* file.

Standalone components make dependencies local and explicit.

3. **Improved tree-shaking / smaller bundles.** Because a standalone component's `imports` array is a precise, statically-analyzable list, the compiler and bundler can eliminate unused code more aggressively than with module-level re-exports.
4. **Simpler lazy loading.** `LoadComponent` lets you code-split at the level of a single component instead of forcing you to manufacture a module purely to get a lazy-loadable boundary.
5. **Ecosystem convergence with React/Vue mental models.** Competing frameworks don't require a "module" concept for basic component composition; Angular's steering committee wanted onboarding parity.
6. **NgModules aren't gone — they're optional.** This is a common interview trap: standalone does **not** deprecate `NgModule`. Modules still exist, are still supported, and are required for a few legacy libraries. Angular's official stance is "NgModules are stable and supported indefinitely," but new code should default to standalone.

1.3 Where This Fits in the Angular Timeline

- **v14:** standalone API released behind developer preview (`standalone: true` required explicitly).
- **v15:** standalone marked stable/production-ready; `provideRouter`, `provideHttpClient`, `importProvidersFrom` shipped.
- **v16:** `Router` standalone directives (`RouterOutlet`, `RouterLink`) commonly used with standalone; `ApplicationConfig` pattern solidified.
- **v17:** CLI defaults new projects/components to standalone; `standalone: true` no longer needs to be written but is still the value.
- **v18/v19:** Signals-based APIs mature alongside standalone; `standalone: true` becomes implicit default in schematics — omitted from generated code.

2. Core Concepts

2.1 The `standalone` Flag and Self-Sufficiency

A standalone component, directive, or pipe declares `standalone: true` (or, from v19 onward, simply omits the `NgModule` declaration entirely since standalone is default) in its decorator metadata. This tells the compiler: "do not expect this class to be declared in any `NgModule`; it is compilable on its own."

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-badge',
  standalone: true, // explicit in v14-v18; implicit/default from v19+
  template: `<span class="badge">{{ label }}</span>`,
  styles: [`.badge { padding: 2px 8px; border-radius: 4px; background: #eee; }`]
})
export class BadgeComponent {
  label = 'New';
}
```

2.2 The `imports` Array on a Component

Because there's no module to supply `CommonModule`, `FormsModule`, or other components, each standalone component lists its own template dependencies in an `imports` array. This is the direct analogue of an `NgModule`'s `imports` + `declarations` combined, scoped to just this one component.

```
import { Component } from '@angular/core';
import { NgIf, NgFor } from '@angular/common';
import { BadgeComponent } from './badge.component';

@Component({
  selector: 'app-user-list',
  standalone: true,
  imports: [NgIf, NgFor, BadgeComponent], // exactly what THIS template needs
  template: `
    <ul *ngIf="users.length">
      <li *ngFor="let u of users">{{ u.name }} <app-badge [label]="u.role"></app-badge></li>
    </ul>
  `
})
export class UserListComponent {
  users = [{ name: 'Asha', role: 'Admin' }];
}
```

Rules governing what can go in `imports`:

- Other **standalone** components, directives, pipes.
- Entire **NgModules** (for interop — e.g., importing `FormsModule`, which is still an `NgModule`). Standalone components can freely import `NgModules`; this is the primary bridge for using non-migrated libraries.
- **Cannot** import a component/directive/pipe that is declared in an `NgModule`'s `declarations` array but not exported by a module you import — same visibility rule as before, just simplified.

A non-standalone component (`standalone: false`, or the legacy default pre-v19 when omitted) **cannot** have an `imports` array — the compiler will error, because such a component's dependencies still come from its owning `NgModule`.

2.3 `bootstrapApplication`

Instead of bootstrapping a root `AppModule` via `platformBrowserDynamic().bootstrapModule(AppModule)`, a standalone app bootstraps a root **component** directly.

```
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';
import { appConfig } from './app/app.config';

bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

`bootstrapApplication` returns a `Promise<ApplicationRef>`. Internally it creates a **platform** (same `platformBrowser()` used before) and an **environment injector** rooted at the application level, then instantiates `AppComponent` as the root view — there is no `AppModule`, no `bootstrap: []` array, no `entryComponents`.

2.4 ApplicationConfig and the providers Array

Root-level providers, which used to live in `AppModule`'s `providers` array (or be `provideIn: 'root'` on the service), are now supplied via an `ApplicationConfig` object passed as the second argument to `bootstrapApplication`.

```
// app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { provideAnimations } from '@angular/platform-browser/animations';
import { routes } from './app.routes';
import { authInterceptor } from './auth.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(withInterceptors([authInterceptor])),
    provideAnimations(),
  ],
};
```

Each `providexxx()` function is a **feature provider factory** — it returns an array of `Provider` objects (often using multi-providers and `ENVIRONMENT_INITIALIZER` tokens internally) that configure a library the same way an `NgModule`'s `forRoot()` used to. This is the standalone replacement for `RouterModule.forRoot(routes)`, `HttpClientModule`, `BrowserAnimationsModule`, etc.

Common provider functions to know:

- `provideRouter(routes, ...features)` — replaces `RouterModule.forRoot()`. Features: `withComponentInputBinding()`, `withDebugTracing()`, `withPreloading()`, `withViewTransitions()`.
- `provideHttpClient(...features)` — replaces `HttpClientModule`. Features: `withInterceptors()`, `withInterceptorsFromDi()`, `withFetch()`, `withXsrfConfiguration()`.
- `provideAnimations()` / `provideNoopAnimations()`.
- `provideStore()` (NgRx), `provideEffects()` — third-party libraries adopted the same pattern.
- `importProvidersFrom(SomeNgModule)` — the escape hatch to pull providers out of a *module you can't yet migrate* (e.g., a third-party module that only exposes providers via `forRoot()` and hasn't shipped a standalone equivalent).

2.5 Standalone Directives and Pipes

Directives and pipes get the same treatment — `standalone: true` and no need for a hosting module.

```
import { Directive, ElementRef, Renderer2 } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
  standalone: true,
})
export class HighlightDirective {
  constructor(private el: ElementRef, private renderer: Renderer2) {
    this.renderer.setStyle(this.el.nativeElement, 'background', '#fff8d');
  }
}
```

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'truncate',
  standalone: true,
})
export class TruncatePipe implements PipeTransform {
  transform(value: string, limit = 20): string {
    return value.length > limit ? value.slice(0, limit) + '...' : value;
  }
}
```

They're consumed exactly like components — added to the `imports` array of whichever standalone component's template uses them:

```
@Component({
  selector: 'app-card',
  standalone: true,
  imports: [HighlightDirective, TruncatePipe],
  template: `

{{ text | truncate:30 }}

`
})
export class CardComponent {
  text = 'A fairly long piece of text that will get truncated';
}
```

2.6 Routing in a Standalone App

Route configs become plain arrays of `Route` objects, and lazy loading a *component* directly (no wrapper module needed) is the headline win:

```
// app.routes.ts
import { Routes } from '@angular/router';

export const routes: Routes = [
  { path: '', loadComponent: () => import('./home/home.component').then(m =>
m.HomeComponent) },
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES),
    providers: [ /* route-level providers, scoped to this subtree only */ ],
  },
];
```

Note the `providers` property directly on a `Route` — this is a standalone-era feature (v14.1+) giving you a *per-route* injector scope without needing a lazy `NgModule` to hang providers off of.

3. Code Examples

3.1 A Complete Minimal Standalone App

```
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';
import { appConfig } from './app/app.config';

bootstrapApplication(AppComponent, appConfig).catch((err) => console.error(err));
```

```
// app/app.component.ts
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { NavbarComponent } from './navbar/navbar.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, NavbarComponent],
  template: `
    <app-navbar></app-navbar>
    <main><router-outlet></router-outlet></main>
  `,
})
export class AppComponent {}
```

```
// app/app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { provideZoneChangeDetection } from '@angular/core';
import { routes } from './app.routes';
```

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(),
  ],
};
```

3.2 Standalone Component with Local + Root Providers

```
import { Component, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { AsyncPipe, NgFor } from '@angular/common';
import { Observable } from 'rxjs';

interface Product { id: number; name: string; }

@Component({
  selector: 'app-product-list',
  standalone: true,
  imports: [NgFor, AsyncPipe],
  // Component-level providers: a fresh instance per component instance,
  // scoped to this component's element injector subtree.
  providers: [],
  template: `
    <ul>
      <li *ngFor="let p of products$ | async">{{ p.name }}</li>
    </ul>
  `,
})
export class ProductListComponent {
  private http = inject(HttpClient);
  products$: Observable<Product[]> = this.http.get<Product[]>('/api/products');
}
```

3.3 Testing a Standalone Component (TestBed simplification)

```
import { TestBed } from '@angular/core/testing';
import { ProductListComponent } from './product-list.component';
import { HttpClientTestingModule } from '@angular/common/http/testing';

describe('ProductListComponent', () => {
  it('creates', async () => {
    await TestBed.configureTestingModule({
      imports: [ProductListComponent, HttpClientTestingModule], // component itself goes in
      // imports, not declarations
    }).compileComponents();

    const fixture = TestBed.createComponent(ProductListComponent);
    expect(fixture.componentInstance).toBeTruthy();
  });
});
```

```
});
```

Note the shift: standalone components go into `TestBed`'s `imports`, never `declarations` (attempting to put a standalone component into `declarations` throws a compile-time/runtime error).

3.4 Interop: Using an NgModule-based Library from a Standalone Component

```
import { Component } from '@angular/core';
import { MatButtonModule } from '@angular/material/button'; // still an NgModule
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-legacy-interop',
  standalone: true,
  imports: [MatButtonModule, FormsModule], // NgModules can be dropped straight into
  `imports`
  template: `<button mat-raised-button [(ngModel)]="value">Click</button>`,
})
export class LegacyInteropComponent {
  value = '';
}
```

3.5 Using a Standalone Component Inside a Legacy NgModule

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { BadgeComponent } from './badge.component'; // standalone

@NgModule({
  declarations: [/* legacy, non-standalone components only */],
  imports: [CommonModule, BadgeComponent], // standalone components go in `imports`, not
  `declarations`
  exports: [BadgeComponent],
})
export class LegacyFeatureModule {}
```

4. Internal Working

4.1 How the Compiler Resolves the Dependency Graph Without NgModules

In the pre-standalone (NgModule-only) world, the Angular compiler (Ivy) computed each component's **compilation scope** by:

1. Finding the `NgModule` that declares the component.
2. Walking that module's `imports` (recursively resolving any re-exported modules) to build the set of directives/pipes/components visible to templates in that module.
3. Caching that resolved scope per module so every component in the module shares it.

With standalone components, Ivy performs the *same conceptual step* — computing a "compilation scope" (the set of selectors resolvable in a template) — but the scope is attached to **the component itself** rather than to a module:

1. The compiler reads the component's own `imports` array.
2. For every entry that is itself standalone (component/directive/pipe), it's added directly to the scope.
3. For every entry that is an `NgModule`, the compiler recursively flattens that module's exports (same algorithm as before) into the scope.
4. The resulting **standalone injector scope + compilation scope** is memoized on the component definition (`ɵcmp`) itself, generated as static metadata (`ɵcmp.dependencies` in the compiled output), so there is no runtime module lookup — it's resolved at compile time exactly like `NgModule`-based apps, just keyed differently.

This means standalone doesn't change *when* resolution happens (compile time, via the Ivy template type checker and compiler) — it changes the *unit of ownership* of that resolved scope from "module" to "component." The generated JS for a standalone component includes something conceptually like:

```
ɵdefineComponent({
  type: UserListComponent,
  selectors: [["@app-user-list"]],
  standalone: true,
  features: [ɵɵStandaloneFeature], // marks the runtime to treat this component specially
  dependencies: [NgIf, NgFor, BadgeComponent], // flattened, resolved imports baked in at
  compile time
  ...
});
```

The `ɵɵStandaloneFeature` is the key runtime hook: it tells Angular's component factory logic that this component brings **its own injector configuration** (see below) rather than inheriting one purely from an enclosing module.

4.2 Environment Injectors in a Standalone App

Angular's DI system has (since Ivy) two injector hierarchies:

- **Element injector hierarchy** — one injector per component/directive host element, following the DOM/component tree (this predates and is orthogonal to standalone).
- **Environment injector hierarchy** — the hierarchy formerly implicit in `NgModules` (`ModuleInjector`), now explicitly named `EnvironmentInjector`. This is where `providedIn: 'root'`, `providedIn: 'platform'`, module/route-level providers, and `bootstrapApplication`'s `providers` array live.

In a standalone app, the environment injector hierarchy looks like:

```
Platform Injector (provided by platformBrowser(), shared even across multiple apps on one
page)
└─ Root Environment Injector (created by bootstrapApplication from
ApplicationConfig.providers)
    └─ Route-level Environment Injector (created lazily when a route with its own
`providers` or lazy-loaded `loadChildren` activates)
        └─ (further nested route environment injectors for nested lazy routes)
```

Each standalone component does **not** automatically get its own environment injector — only:

- The **root** (`bootstrapApplication` call).
- **Lazy-loaded routes** that specify `providers` on the `Route`, or that lazy-load a `loadChildren` route config (the Router creates a child `EnvironmentInjector` for that boundary, mirroring what a lazy `NgModule` used to give you for free).
- Explicit calls to `createEnvironmentInjector()` (e.g., for imperative/dynamic component creation scenarios, `ViewContainerRef.createComponent()` with a custom injector).

Regular component-level `providers` (declared in `@Component({ providers: [...] })`) still create an **element injector**, not an environment injector — that behavior is unchanged from the `NgModule` era and is orthogonal to standalone.

This is why the classic "lazy module gives you a new injector scope for free" pattern is replicated in standalone apps via `providers` on a lazy `Route` rather than via a lazy `NgModule` — the Router explicitly creates an `EnvironmentInjector` for that route boundary using `createEnvironmentInjector(route.providers, parentEnvironmentInjector)`.

4.3 `bootstrapApplication` Internals (Simplified)

Roughly, `bootstrapApplication(RootComponent, config)`:

1. Calls `createPlatform()` / reuses the existing platform (`platformBrowser()`), which is itself an injector holding platform-wide singletons (e.g., `DOCUMENT`, `PLATFORM_ID`).
2. Creates a root `EnvironmentInjector` whose providers are `config.providers` merged with Angular's own core providers (change detection scheduler, `ApplicationRef`, `NgZone`, etc.) — this replaces what `AppModule`'s `providers` + `BrowserModule`'s providers used to assemble.
3. Instantiates `RootComponent` as if it were the module's `bootstrap` component, attaching it to the DOM node matching its selector.
4. Returns a `Promise<ApplicationRef>`, same public contract as `bootstrapModule` before it, so tooling (protractor/e2e harnesses, `ApplicationRef.tick()`, etc.) behaves identically.

The important interview point: **standalone did not require inventing a new injector system**. It reused the `EnvironmentInjector` / `ElementInjector` split that Ivy already had (introduced to support tree-shakable providers and `providedIn: 'root'`), and simply gave `bootstrapApplication` a way to seed the root `EnvironmentInjector` without a module.

5. Edge Cases & Gotchas

1. **Mixing standalone and `NgModule` in the same app is fully supported, but the direction of importing matters.** A standalone component can import an `NgModule` (common for third-party libraries). An `NgModule` can declare non-standalone components and separately `import` standalone ones — but it **cannot** put a standalone component in its `declarations` array. Doing so throws: `Component X is standalone, and cannot be declared in an NgModule`.
2. `standalone: false` **still exists and is meaningful**, especially post-v19 where standalone is the default. If you generate a component and need classic `NgModule`-style declaration behavior (e.g., migration in a legacy codebase), you must now explicitly write `standalone: false` — omitting the flag

on v19+ means "standalone: true" by default, which is a common migration trap when copy-pasting older component code that relied on being declared in a module.

3. **Forgetting to add a used directive/pipe/component to `imports`** is now a compile-time (AOT) or template-type-check error rather than a silent runtime "no matching directive" issue in many cases — a real improvement, but it surprises developers used to the module system's more permissive dev-mode behavior (JIT with missing imports sometimes just silently did nothing).
4. **`CommonModule` habits.** Developers used to importing the whole `CommonModule` in every `SharedModule` sometimes import all of `CommonModule` into every standalone component "for safety," defeating some of the fine-grained tree-shaking benefit. Best practice: import only `NgIf`, `NgFor`, `NgClass`, `AsyncPipe`, etc. individually, or rely on the built-in control flow (`@if`, `@for`, `@switch` from v17+) which needs **no import at all**.
5. **`TestBed.configureTestingModule` semantics changed.** Standalone components/directives/pipes go into the `imports` array of the `TestBed` config, never `declarations`. Mixing this up (`declarations: [MyStandaloneComponent]`) throws at test time. Also, `TestBed.overrideComponent` for changing a standalone component's own `imports` requires `TestBed.overrideComponent(Cmp, { set: { imports: [...] } })` rather than fiddling with a module.
6. **`providedIn: 'root'` vs `bootstrapApplication` providers vs route providers — scope confusion.** All three create/contribute to environment injectors, but at different points in the hierarchy. A service registered only via a lazy route's `providers` array is **not** visible to eagerly-loaded components outside that route subtree — this trips up people used to "just put it in `AppModule.providers` and it's everywhere."
7. **`importProvidersFrom()` is a migration crutch, not a long-term pattern.** It lets you pull a module's providers (e.g., a third-party `SomeModule.forRoot(config)`) into `bootstrapApplication`'s `providers` array, but the underlying module and its own imports are still fully instantiated — you get none of the tree-shaking benefit until the library itself ships standalone-first provider functions (`provideSomeLib()`).
8. **Bootstrapping multiple root components / multiple apps on one page** works differently: with `NgModules` you listed multiple entries in `bootstrap: [...]`; with standalone, you call `bootstrapApplication()` once per root component, and each call creates its own root environment injector (though they can share the same `platformBrowser()` platform injector). Forgetting this can lead to duplicated singleton services if you're not careful about `providedIn: 'root'` behavior across multiple bootstrap calls.
9. **Migration schematic (`ng generate @angular/core:standalone`) runs in three steps** — (1) convert declarations to standalone, (2) remove now-empty/redundant `NgModules`, (3) convert `bootstrapModule` to `bootstrapApplication` — and is designed to be run and committed one step at a time, because running all three blindly on a large app frequently produces circular import errors or duplicate-provider warnings that are much harder to bisect after the fact.
10. **Circular imports become more visible.** Because dependencies are declared per-component rather than aggregated in a module, two standalone components that import each other directly (rather than through a shared parent) will surface a circular-import compile error immediately, whereas previously a `SharedModule` might have masked the cycle by being the single import point for both.
11. **Route-level providers create a new environment injector per activation**, not per route config — meaning if the same route is deactivated and reactivated (e.g., navigating away and back), a *new*

instance of any route-scoped service is created, resetting its state. This is easy to miss when migrating a singleton service out of `providedIn: 'root'` into a route's `providers` for "scoping" purposes.

12. **Standalone pipes/directives still require unique selectors/names** app-wide in practice — standalone doesn't relax Angular's usual selector collision rules, it just removes the module indirection for resolving them.

6. Interview Questions & Answers

Q1. What does `standalone: true` actually do to a component?

A: It tells the Angular compiler that this component, directive, or pipe is self-describing — it does not need to be declared inside an `NgModule` to be compiled or used. Instead, the component declares its own template dependencies via an `imports` array. The compiler generates a compilation scope (the list of usable selectors) and, via `ɵɵStandaloneFeature`, an associated injector configuration directly on the component's definition, rather than deriving it from an enclosing module.

Q2. Does adopting standalone components mean NgModules are deprecated?

A: No. This is a common misconception. NgModules remain fully supported and are not deprecated. Standalone is simply the new *default* and recommended style for new code since Angular 17 (and implicit since Angular 19), primarily to reduce boilerplate. Existing NgModule-based apps continue to work indefinitely, and standalone components can freely interoperate with NgModules in both directions (with the one restriction that a standalone component cannot be put into an NgModule's `declarations` array).

Interviewer intent: This question checks whether the candidate conflates "new default" with "old thing removed" — a common misunderstanding that leads to bad migration decisions (e.g., panic-migrating a stable, working NgModule app with no real benefit).

Q3. How do you bootstrap a standalone application, and what replaces `AppModule`?

A: You call `bootstrapApplication(RootComponent, applicationConfig)` from `@angular/platform-browser`, passing the root component directly instead of a root `NgModule`. The `ApplicationConfig` object (typically exported from `app.config.ts`) supplies a `providers` array that replaces what used to live in `AppModule.providers` plus whatever `BrowserModule`, `HttpClientModule`, `RouterModule.forRoot()`, etc. used to contribute. There is no `bootstrap: []` array and no root module class at all.

Q4. What is `provideRouter()` and how does it differ from `RouterModule.forRoot()`?

A: `provideRouter(routes, ...features)` is a standalone-era function that returns an array of providers configuring the Router — the functional equivalent of `RouterModule.forRoot(routes)`. Rather than passing "module features" via nested module imports (e.g., `RouterModule.forRoot(routes, { enableTracing: true })`), you compose *feature functions* like `withComponentInputBinding()`, `withPreloading()`, `withDebugTracing()`, or `withViewTransitions()` as extra arguments. This functional, tree-shakable composition style is the general pattern Angular adopted for all "library setup" (see also `provideHttpClient(withInterceptors(...))`).

Q5. Can a standalone component use an NgModule-based library like Angular Material? How?

A: Yes. A standalone component can import any `NgModule` directly into its own `imports` array — e.g., `imports: [MatButtonModule, MatCardModule]`. The compiler flattens that module's exports into the component's compilation scope exactly as it would when the module was imported by another module. This is the primary interoperability bridge that made incremental adoption of standalone possible without waiting for every third-party library to ship a standalone-first API.

Q6. What happens if you try to put a standalone component into an NgModule's `declarations` array?

A: It's a compile-time/runtime error: Angular explicitly disallows declaring a standalone component/directive/pipe in `declarations`, because standalone components manage their own compilation scope and don't need (and can't accept) module ownership. The correct way to consume a standalone component from within an `NgModule` is to add it to that module's `imports` array instead, and if needed, re-export it from `exports`.

Q7. Explain the difference between an `EnvironmentInjector` and an `ElementInjector` in the context of standalone apps.

A: `ElementInjector`s form a hierarchy mirroring the component/DOM tree — each component/directive instance gets one, populated by its own `@Component({ providers }) / viewProviders` and inherited from ancestor elements; this predates standalone. `EnvironmentInjector`s form the hierarchy that used to be implicit in NgModules (formerly called `ModuleInjector`) — this is where `providedIn: 'root'` services, `bootstrapApplication`'s `ApplicationConfig.providers`, and lazy route `providers` live. Standalone apps still have both hierarchies; they just create environment injectors explicitly at the root (`bootstrapApplication`) and at lazy route boundaries, instead of implicitly at every `NgModule` boundary.

Interviewer intent: Distinguishes candidates who've memorized "standalone removes DI scoping" (wrong) from those who understand DI scoping was decoupled from modules well before standalone existed and simply gained new entry points.

Q8. How do you scope a provider to just one lazy-loaded feature area in a standalone app, without a feature NgModule?

A: Add a `providers` array directly to the `Route` object for that feature (available since Angular 14.1), e.g. `{ path: 'admin', loadChildren: () => import('./admin.routes').then(m => m.ADMIN_ROUTES), providers: [AdminOnlyService] }`. The Router creates a new child `EnvironmentInjector` scoped to that route subtree using those providers, giving the same encapsulation a lazy `NgModule` used to provide — without needing a module at all.

Q9. What is `importProvidersFrom()` for, and why shouldn't it be a permanent solution?

A: `importProvidersFrom(SomeModule)` extracts the provider configuration from an existing `NgModule` (including ones configured via `forRoot()` / `forChild()` -style static methods) so it can be passed into `bootstrapApplication`'s `providers` array or a route's `providers`. It's the standard escape hatch for using libraries that haven't yet published standalone-first provider functions. It's a bridge, not an end-state: the referenced module (and everything *it* imports) is still fully instantiated as a unit, so you don't get the finer-grained tree-shaking that a dedicated `providexxx()` function would give you once the library ships one.

Q10. Walk through what the `ng generate @angular/core:standalone` migration schematic does, step by step, and why it's split into stages.

A: It's a three-step, CLI-driven migration: **Step 1** converts every eligible component/directive/pipe to `standalone: true` and fills in its `imports` array based on static analysis of what the template actually uses. **Step 2** removes NgModules that have become empty or redundant once their declarations are gone (folding remaining providers into either the root config or appropriate scopes). **Step 3** converts `bootstrapModule(AppModule)` to `bootstrapApplication(AppComponent, appConfig)`. It's staged deliberately because each step can independently break the build (missing imports, orphaned providers, circular imports surfacing once module-level aggregation disappears) — running one step at a time and committing/testing between steps makes failures bisectable, whereas running all three in one shot on a nontrivial app usually produces a wall of hard-to-triage errors.

Q11. Your team has a large NgModule-based app and wants to migrate to standalone gradually rather than all at once. Is that possible, and what's the recommended order?

A: Yes — standalone was explicitly designed for incremental adoption. The recommended order (matching what the official schematic and Angular docs suggest) is: (1) convert leaf-level, low-dependency components/directives/pipes to standalone first, since they have the fewest cross-module entanglements; (2) update NgModules to `imports` those newly-standalone pieces instead of `declarations`; (3) progressively work up the tree toward feature and shared modules, converting each once its children are standalone; (4) once `AppModule` itself has nothing left to declare, convert routing to standalone `Routes / LoadComponent`; (5) finally replace `bootstrapModule` with `bootstrapApplication`. Throughout, standalone components can already be consumed by not-yet-migrated NgModules, so the app remains shippable at every intermediate commit.

Interviewer intent: Tests whether the candidate can plan a real, low-risk migration rather than just recite "standalone exists" trivia — a very common on-the-job scenario given how many production Angular apps predate v14.

Q12. Why did Angular choose per-component `imports` arrays instead of, say, a lighter-weight module concept?

A: The core complaint about NgModules wasn't "modules are a bad abstraction" in the abstract — it was that they forced *every* component author to reason about a second, separate declarative surface (the module) just to answer "what can my template use?" By attaching the dependency list directly to the component that consumes it, Angular collapsed two levels of indirection into one, made dependencies locally visible and directly tied to what a template-type-checker or IDE could statically verify, and enabled finer-grained tree-shaking since the compiler no longer needs to reason about a whole module's transitive exports to determine what one component actually needs. NgModules remain available for cases that genuinely benefit from grouping (e.g., configuring a big third-party library with `forRoot()` -style setup), but they're no longer *mandatory* scaffolding for basic component composition.

Q13. What breaks if you forget to add `CommonModule / NgIf` to a standalone component's `imports` but use `*ngIf` in its template?

A: With Ivy's AOT template type checking (which is on by default and stricter for standalone components), this surfaces as a compile-time error — something like "the `*ngIf` structural directive was used, but no directive with matching selector `ngIf` was found." This is actually a callable improvement over the old NgModule world's dev-mode behavior, where a similar mistake could sometimes fail more obscurely at runtime (or, in JIT mode, silently do nothing) rather than being caught immediately at build time.

Q14. Can a standalone component have its own `providers`, and how does that differ from a service registered via `providexxx()` in `bootstrapApplication`?

A: Yes — `@Component({ providers: [...] })` on a standalone component works exactly as it always has: it creates an **element injector** entry scoped to that component instance and its view/content children, giving every instance of the component (potentially) its own instance of the service if the service isn't already provided higher up. This is unrelated to standalone-ness and predates it. In contrast, providers supplied via `bootstrapApplication`'s `ApplicationConfig.providers` (or a `providexxx()` call inside it) register into the root **environment injector**, making the service effectively app-wide/singleton (subject to normal injector hierarchy resolution), analogous to what used to be `AppModule.providers`.

7. Quick Revision Cheat Sheet

- `standalone: true` → component/directive/pipe needs no owning `NgModule`; default since Angular 19 (flag can be omitted; write `standalone: false` to opt out).

- `imports: [...]` on a component → replaces the module's `declarations` + `imports` for that component's template scope. Can contain standalone components/directives/pipes **and/or** whole NgModules.
- `bootstrapApplication(RootComponent, appConfig)` → replaces `bootstrapModule(AppModule)`; no root module, no `bootstrap: []`.
- `ApplicationConfig.providers` → replaces `AppModule.providers` + module-level global setup (`BrowserModule`, `HttpClientModule`, `RouterModule.forRoot()`, etc.).
- `provideRouter(routes, ...features)`, `provideHttpClient(...features)`, `provideAnimations()` → functional, tree-shakable replacements for the old `*Module.forRoot()` pattern; compose with `withxxx()` feature functions.
- `importProvidersFrom(NgModule)` → migration bridge to pull providers out of a not-yet-standalone library module.
- Standalone component in an `NgModule` → goes in `imports`, **never** `declarations`.
- `NgModule` in a standalone component → goes in that component's `imports` array directly (interop works).
- **Route-level providers** on a `Route` object → creates a scoped `EnvironmentInjector` for that route subtree, replacing what a lazy feature `NgModule` used to give for free.
- Two injector hierarchies, unchanged in *kind* by standalone: `ElementInjector` (component/DOM tree, via `@Component({ providers })`) vs `EnvironmentInjector` (root/route/module scope, via `providedIn: 'root'`, `bootstrapApplication` providers, route providers).
- `TestBed`: standalone components go in `imports`, not `declarations`.
- Migration: `ng generate @angular/core:standalone` — 3 staged steps (convert to standalone → drop empty modules → switch to `bootstrapApplication`); run and verify one stage at a time.
- NgModules are **not deprecated** — standalone is the new default/recommended style, not a mandatory rewrite.

Created By - Durgesh Singh